Group Relative Policy Optimization (GRPO)

CS 5624: Natural Language Processing

Fall 2026

https://tuvllms.github.io/nlp-fall-2026/

Tu Vu

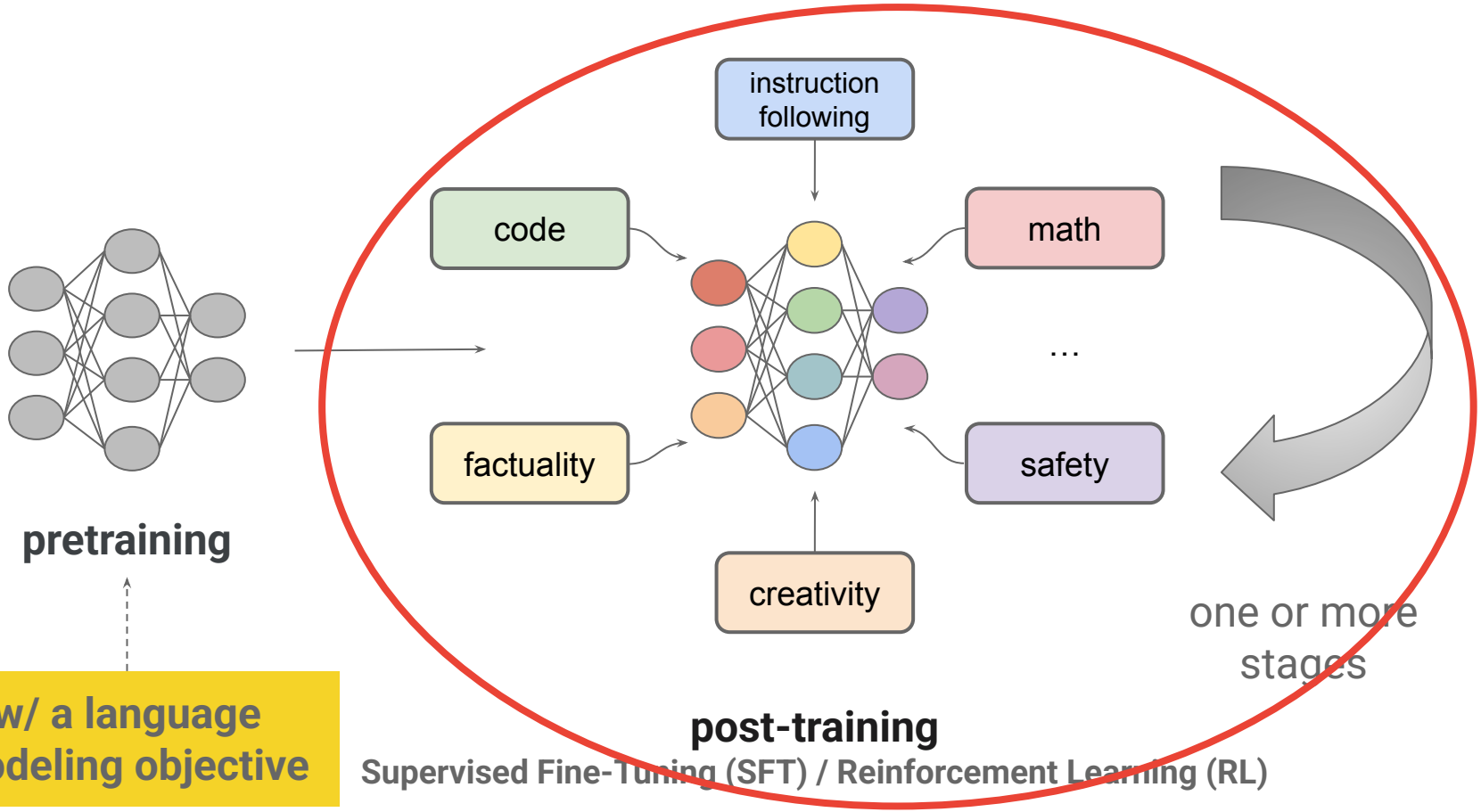
VIRGINIA TECH

Logistics

- Quiz2, HW2 coming

AI news

Where are we?


instruction following
code
math
...
factuality
safety
creativity
one or more stages
pretraining
w/ a language modeling objective
post-training
Supervised Fine-Tuning (SFT) / Reinforcement Learning (RL)

Recap: KL divergence

Assume two different distributions for predicting the next word:

- P (from Model 1):
  - *mat* → 0.7
  - *floor* → 0.2
  - *chair* → 0.1
- Q (from Model 2):
  - *mat* → 0.5
  - *floor* → 0.3
  - *chair* → 0.2

Kullback–Leibler (KL) Divergence Calculation

KL divergence measures how much P diverges from Q:

$$D_{KL}(P||Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}$$

Substituting the values:

$$D_{KL}(P||Q) = 0.7 \log \frac{0.7}{0.5} + 0.2 \log \frac{0.2}{0.3} + 0.1 \log \frac{0.1}{0.2}$$

$$D_{\mathrm{KL}}(P\|Q) = \sum_{y} P(y) \log \frac{P(y)}{Q(y)} = \mathbb{E}_{y\sim P}\left[\log \frac{P(y)}{Q(y)}\right]$$

$$D_{\mathrm{KL}}(\pi\|\pi_{\mathrm{ref}}) = \sum_{y} \pi(y \mid x) \log \frac{\pi(y \mid x)}{\pi_{\mathrm{ref}}(y \mid x)} = \mathbb{E}_{y\sim\pi(\cdot|x)}\left[\log \frac{\pi(y \mid x)}{\pi_{\mathrm{ref}}(y \mid x)}\right]$$

DPO vs. RLHF

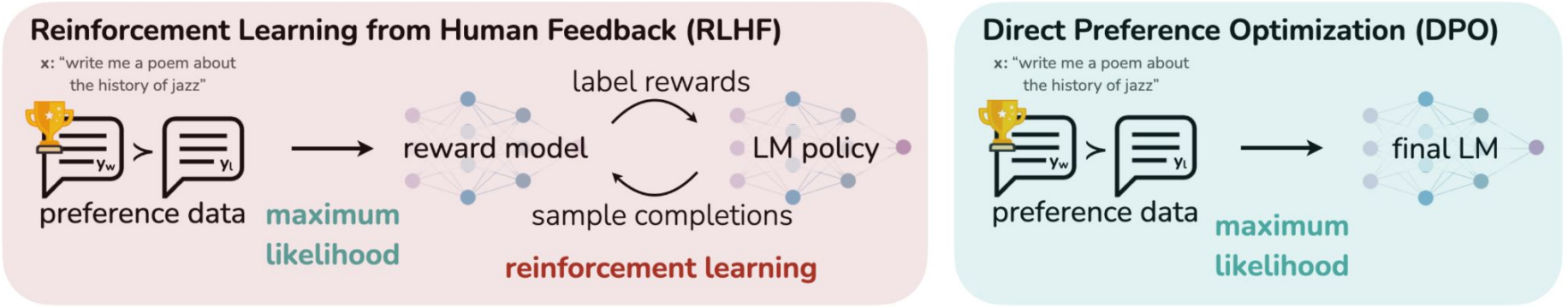


Figure 1: **DPO optimizes for human preferences while avoiding reinforcement learning.** Existing methods for fine-tuning language models with human feedback first fit a reward model to a dataset of prompts and human preferences over pairs of responses, and then use RL to find a policy that maximizes the learned reward. In contrast, DPO directly optimizes for the policy best satisfying the preferences with a simple classification objective, fitting an *implicit* reward model whose corresponding optimal policy can be extracted in closed form.

Direct Preference Optimization (DPO)

$$\max_{\pi} \mathbb{E}_{x\sim D, y\sim\pi(y|x)} \left[r(x,y) \right] - \beta D_{\mathrm{KL}} \left[\pi(y|x) \,||\, \pi_{\mathrm{ref}}(y|x) \right]$$

$$= \min_{\pi} \mathbb{E}_{x \sim D} \mathbb{E}_{y \sim \pi(y|x)} \left[\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} - \frac{1}{\beta} r(x, y) \right]$$

$$= \min_{\pi} \mathbb{E}_{x \sim D} \left[\mathbb{E}_{y \sim \pi(y|x)} \left[\log \frac{\pi(y|x)}{\pi^*(y|x)} \right] - \log Z(x) \right]$$

$$= \min_{\pi} \mathbb{E}_{x \sim D} \left[D_{KL}(\pi(y|x) || \pi^*(y|x)) - \log Z(x) \right]$$

$$\pi(y|x) = \pi^*(y|x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y|x) \exp\left(\frac{1}{\beta} r(x, y) \right)$$

$$Z(x) = \sum_{y} \pi_{\text{ref}}(y|x) \exp\left(\frac{1}{\beta} r(x, y) \right)$$

DPO objective under the Bradley-Terry model

$$\pi^*(y \mid x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y \mid x) \exp\left(\frac{1}{\beta} r^*(x, y)\right)$$

$$r^*(x, y) = \beta \log \frac{\pi^*(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x)$$

DPO objective under the Bradley-Terry model (cont'd)

$$p^*(y_w \succ y_l \mid x) = \frac{\exp(r^*(x, y_w))}{\exp(r^*(x, y_w)) + \exp(r^*(x, y_l))}$$

$$= \sigma\left(r^*(x, y_w) - r^*(x, y_l)\right)$$

DPO objective under the Bradley-Terry model (cont'd)

$$p^*(y_w \succ y_l \mid x) = \frac{\exp(r^*(x, y_w))}{\exp(r^*(x, y_w)) + \exp(r^*(x, y_l))}$$

$$= \sigma\left(r^*(x, y_w) - r^*(x, y_l)\right)$$

$$= \sigma\left(\beta \log \frac{\pi^*(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi^*(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)$$

DPO objective under the Bradley-Terry model (cont'd)

$$\max_{\pi_\theta} \; p(y_w \succ y_l \mid x) = \max_{\pi_\theta} \; \sigma(r^*(x, y_w) - r^*(x, y_l))$$

$$L_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim D} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} \right) \right]$$

DPO moves toward the RLHF optimum because every preference pair tells us a pairwise relationship that the optimal RLHF policy must satisfy, and the DPO loss directly pushes the policy toward satisfying those relationships.

DPO objective under the Bradley-Terry model (cont'd)

The gradient with respect to the parameters θ can be written as:

$$\nabla_\theta \mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) =$$
$$-\beta \mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\Big[\underbrace{\sigma(\hat{r}_\theta(x, y_l) - \hat{r}_\theta(x, y_w))}_{\text{higher weight when reward estimate is wrong}}\Big[\underbrace{\nabla_\theta \log \pi(y_w \mid x)}_{\text{increase likelihood of } y_w} - \underbrace{\nabla_\theta \log \pi(y_l \mid x)}_{\text{decrease likelihood of } y_l}\Big]\Big],$$

where $\hat{r}_\theta(x, y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$ is the reward implicitly defined by the language model π_θ and reference model π_{ref} (more in Section 5).

SFT

The tokens come from a **human/reference response**:

$$o = (o_1, \ldots, o_T) \sim \text{human data}.$$

We maximize

$$\boxed{J_{\text{SFT}}(\theta) = \frac{1}{T} \sum_{t=1}^{T} \log \pi_\theta(o_t \mid o_{<t})}$$

So the model is told:

"Make the human's tokens more likely."

RL

The tokens are **sampled from the model**:

$$o = (o_1, \ldots, o_T) \sim \pi_{\theta_{\text{old}}}(\cdot \mid x)$$

Then we evaluate the generated response, get reward r, and derive advantage A.

Conceptually, the objective is

$$\boxed{J_{\text{RL}}(\theta) = \frac{1}{T} \sum_{t=1}^{T} A \log \pi_\theta(o_t \mid x, o_{<t})}$$

Thus:

$$A > 0 \quad \Rightarrow \quad \text{make the model's sampled tokens more likely}$$

and

$$A < 0 \quad \Rightarrow \quad \text{make the model's sampled tokens less likely.}$$

Ignoring the reference model for one moment, the intuition is

$$\max_{\theta} \; \sigma\left(\log \pi_\theta(y_w \mid x) - \log \pi_\theta(y_l \mid x)\right).$$

But an autoregressive LM factorizes each response:

$$\log \pi_\theta(y_w \mid x) = \sum_{t=1}^{T_w} \log \pi_\theta(y_{w,t} \mid x, y_{w,<t})$$

$$\log \pi_\theta(y_l \mid x) = \sum_{t=1}^{T_l} \log \pi_\theta(y_{l,t} \mid x, y_{l,<t}).$$

So at the token level, DPO is comparing

$$\underbrace{\sum_t \log \pi_\theta(y_{w,t} \mid x, y_{w,<t})}_{\text{preferred tokens}} - \underbrace{\sum_t \log \pi_\theta(y_{l,t} \mid x, y_{l,<t})}_{\text{rejected tokens}}.$$

The reference model adds a baseline to both sides:

$$\max_{\theta} \; \log \sigma \left[\beta \left(\sum_t \log \frac{\pi_\theta(y_{w,t} \mid x, y_{w,<t})}{\pi_{\text{ref}}(y_{w,t} \mid x, y_{w,<t})} - \sum_t \log \frac{\pi_\theta(y_{l,t} \mid x, y_{l,<t})}{\pi_{\text{ref}}(y_{l,t} \mid x, y_{l,<t})} \right) \right].$$

So what is DPO telling each token?

In the simplified view:

$$y_{w,t} \quad \Rightarrow \quad \text{make the preferred response's tokens more likely}$$

$$y_{l,t} \quad \Rightarrow \quad \text{make the rejected response's tokens less likely.}$$

This gives a nice comparison with your SFT/RL slide:

	Tokens come from	Token-level signal
SFT	Human/reference response	↑ human tokens
RL	Model-generated response	$A > 0 :\uparrow$, $A < 0 :\downarrow$
DPO	Preferred + rejected responses	↑ preferred tokens, ↓ rejected tokens

DPO says: deviate from the reference model enough to better match the preferences, but use the reference model as an anchor.

arXiv:2404.10719v3 [cs.CL] 10 Oct 2024

Is DPO Superior to PPO for LLM Alignment? A Comprehensive Study

Shusheng Xu [1] **Wei Fu** [1] **Jiaxuan Gao** [1] **Wenjie Ye** [2] **Weilin Liu** [2]
Zhiyu Mei [1] **Guangju Wang** [2] **Chao Yu** [*1] **Yi Wu** [*1 2 3]

Abstract

Reinforcement Learning from Human Feedback (RLHF) is currently the most widely used method to align large language models (LLMs) with human preferences. Existing RLHF methods can be roughly categorized as either *reward-based* or *reward-free*. Novel applications such as ChatGPT and Claude leverage *reward-based* methods that first learn a reward model and apply actor-critic algorithms, such as Proximal Policy Optimization (PPO). However, in academic benchmarks, the state-of-the-art results are often achieved via *reward-free* methods, such as Direct Preference Optimization (DPO). *Is DPO truly superior to PPO? Why does PPO perform poorly on these benchmarks?* In this paper, we first conduct both theoretical and empirical

2023). To leverage the formidable capabilities of LLMs in practical applications, a growing amount of research has underscored the importance of aligning these models with human preferences (Agrawal et al., 2023; Kadavath et al., 2022; Shi et al., 2023; Liang et al., 2021; Sheng et al., 2019). Various methods have been developed for fine-tuning LLMs, with popular approaches including Supervised Fine-Tuning (SFT) (Peng et al., 2023) and Reinforcement Learning from Human Feedback (RLHF) (Ziegler et al., 2019; Stiennon et al., 2020; Ouyang et al., 2022). Typically, fine-tuning involves two phases: SFT to establish a base model, followed by RLHF for enhanced performance. SFT involves imitating high-quality demonstration data, while RLHF refines LLMs through preference feedback.

Within RLHF, two prominent approaches are *reward-based* and *reward-free* methods. Reward-based methods, pioneered by OpenAI (Ouyang et al., 2022; Ziegler et al., 2019; Stiennon et al., 2020), construct a reward model using pref-

Preference Fine-Tuning of LLMs Should Leverage Suboptimal, On-Policy Data

Fahim Tajwar[1*], **Anikait Singh**[2*], **Archit Sharma**[2], **Rafael Rafailov**[2], **Jeff Schneider**[1], **Tengyang Xie**[4], **Stefano Ermon**[2], **Chelsea Finn**[2] **and Aviral Kumar**[3]

[*]Equal contributions (ordered via coin-flip), [1]Carnegie Mellon University, [2]Stanford University, [3]Google DeepMind, [4]UW-Madison

Learning from preference labels plays a crucial role in fine-tuning large language models. There are several distinct approaches for preference fine-tuning, including supervised learning, on-policy reinforcement learning (RL), and contrastive learning. Different methods come with different implementation tradeoffs and performance differences, and existing empirical findings present different conclusions, for instance, some results show that online RL is quite important to attain good fine-tuning results, while others find (offline) contrastive or even purely supervised methods sufficient. This raises a natural question: ***what kind of approaches are important for fine-tuning with preference data and why?*** In this paper, we answer this question by performing a rigorous analysis of a number of fine-tuning techniques on didactic and full-scale LLM problems. Our main finding is that, in general, approaches that use on-policy sampling or attempt to push down the likelihood on certain responses (i.e., employ a "negative gradient") outperform offline and maximum likelihood objectives. We conceptualize our insights and unify methods that use on-policy sampling or negative gradient under a notion of mode-seeking objectives for categorical distributions. Mode-seeking objectives are able to alter probability mass on specific bins of a categorical distribution at a fast rate compared to maximum likelihood, allowing them to relocate masses across bins more effectively. Our analysis prescribes actionable insights for preference fine-tuning of LLMs and informs how data should be collected for maximal improvement.

[cs.LG] 2 Jun 2024

Group Relative Policy Optimization (GRPO)

3300v3 [cs.CL] 27 Apr 2024

deepseek

DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models

Zhihong Shao[1,2*†], Peiyi Wang[1,3*†], Qihao Zhu[1,3*†], Runxin Xu[1], Junxiao Song[1]
Xiao Bi[1], Haowei Zhang[1], Mingchuan Zhang[1], Y.K. Li[1], Y. Wu[1], Daya Guo[1*]

[1]DeepSeek-AI, [2]Tsinghua University, [3]Peking University

{zhihongshao,wangpeiyi,zhuqh,guoday}@deepseek.com
https://github.com/deepseek-ai/DeepSeek-Math

Abstract

Mathematical reasoning poses a significant challenge for language models due to its complex and structured nature. In this paper, we introduce DeepSeekMath 7B, which continues pre-training DeepSeek-Coder-Base-v1.5 7B with 120B math-related tokens sourced from Common Crawl, together with natural language and code data. DeepSeekMath 7B has achieved an impressive score of 51.7% on the competition-level MATH benchmark without relying on external toolkits and voting techniques, approaching the performance level of Gemini-Ultra and GPT-4. Self-consistency over 64 samples from DeepSeekMath 7B achieves 60.9% on MATH. The mathematical reasoning capability of DeepSeekMath is attributed to two key factors: First, we harness the significant potential of publicly available web data through a meticulously engineered data selection pipeline. Second, we introduce Group Relative Policy Optimization (GRPO), a variant of Proximal Policy Optimization (PPO), that enhances mathematical reasoning abilities while concurrently optimizing the memory usage of PPO.

1. Motivation

PPO requires a **value model** to compute advantages.

Problems:

- training the value model is expensive
- value estimates can be unstable
- it doubles model size during training

GRPO removes the value model entirely.

> Instead of learning a value baseline, GRPO uses **relative rewards within a group of sampled completions**.

GRPO (cont'd)

2. Basic idea

For a prompt x, sample **multiple completions**:

$$y^{(1)}, y^{(2)}, \ldots, y^{(K)}$$

Each completion gets a reward:

$$R^{(1)}, R^{(2)}, \ldots, R^{(K)}$$

Now compute a **group-normalized advantage**.

GRPO (cont'd)

3. GRPO advantage

First compute group statistics:

$$\mu = \frac{1}{K}\sum_{k=1}^{K} R^{(k)}$$

$$\sigma = \sqrt{\frac{1}{K}\sum_{k=1}^{K}(R^{(k)} - \mu)^2}$$

Then define advantage:

$$A^{(k)} = \frac{R^{(k)} - \mu}{\sigma}$$

So each completion is compared **relative to the group**.

Prompt A

$$[8, 9, 9, 10]$$

Now multiply **every reward by 50**:

Prompt B

$$[400, 450, 450, 500]$$

Consider the best response in each group.

RLOO for Prompt A:

$$10 - \frac{8 + 9 + 9}{3} = \boxed{1.33}$$

RLOO for Prompt B:

$$500 - \frac{400 + 450 + 450}{3} = \boxed{66.67}$$

But for **GRPO**:

$$A_A = \frac{10 - 9}{0.71} \approx \boxed{1.41}$$

For Prompt B, both the difference from the mean **and the standard deviation** increase by 50:

$$A_B = \frac{500 - 450}{35.36} \approx \boxed{1.41}$$

GRPO (cont'd)

GRPO still uses the **PPO clipped objective**, but the advantage is sequence-level.

For each token:

$$L^{GRPO} = \sum_t \min\left(\rho_t A^{(k)}, \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon) A^{(k)}\right)$$

where

$$\rho_t = \frac{\pi_\theta(a_t|s_t)}{\pi_{\text{old}}(a_t|s_t)}$$

Comparison of Policy Gradient Methods (with KL)

Method	Advantage definition	Token-level advantage?	Value model
REINFORCE	$A = R - b$	❌ No	❌ No
RLOO	$A^{(k)} = R^{(k)} - \frac{1}{K-1}\sum_{i \neq k} R^{(i)}$	❌ No	❌ No
PPO	A_t from TD / GAE	✅ Yes	✅ Yes
GRPO	$A^{(k)} = \frac{R^{(k)} - \mu}{\sigma}$	❌ No	❌ No

Comparison of Policy Gradient Methods (with KL)

Method	Policy objective per token L_t	KL usage	Key idea
REINFORCE	$L_t = \log \pi_\theta(a_t \mid s_t)\, A$	Reward shaping: $R \leftarrow R - \beta \sum_t \mathrm{KL}_t$	Reinforce tokens of high-reward sequences
RLOO	$L_t = \log \pi_\theta(a_t \mid s_t)\, A^{(k)}$	Same reward shaping with KL	Compare each completion with others
PPO	$L_t = \min(\rho_t A_t,\ \mathrm{clip}(\rho_t, 1-\epsilon, 1+\epsilon) A_t)$	Usually per-token reward: $r_t = -\beta \mathrm{KL}_t$	Stable updates + token-level credit
GRPO	$L_t = \min(\rho_t A^{(k)},\ \mathrm{clip}(\rho_t, 1-\epsilon, 1+\epsilon) A^{(k)})$	Same per-token KL penalty	PPO-style update without value model

GRPO (cont'd)

$$J(\theta) = \frac{1}{G}\sum_{i=1}^{G}\frac{1}{|a_i|}\sum_{t=1}^{|a_i|}\left(\min\left(\frac{\pi_\theta(a_{i,t}|s_i)}{\pi_{\theta_{\text{old}}}(a_{i,t}|s_i)}A_{i,t},\ \text{clip}\left(\frac{\pi_\theta(a_{i,t}|s_i)}{\pi_{\theta_{\text{old}}}(a_{i,t}|s_i)}, 1-\varepsilon, 1+\varepsilon\right)A_{i,t}\right)\right.$$
$$\left. - \beta\mathcal{D}_{\text{KL}}(\pi_\theta(\cdot|s_i)\|\pi_{\text{ref}}(\cdot|s_i))\right)$$

With the advantage computation for the completion index i:

$$A_i = \frac{r_i - \text{mean}(r_1, r_2, \cdots, r_G)}{\text{std}(r_1, r_2, \cdots, r_G)}.$$

Group Relative Policy Optimization (GRPO) (cont'd)

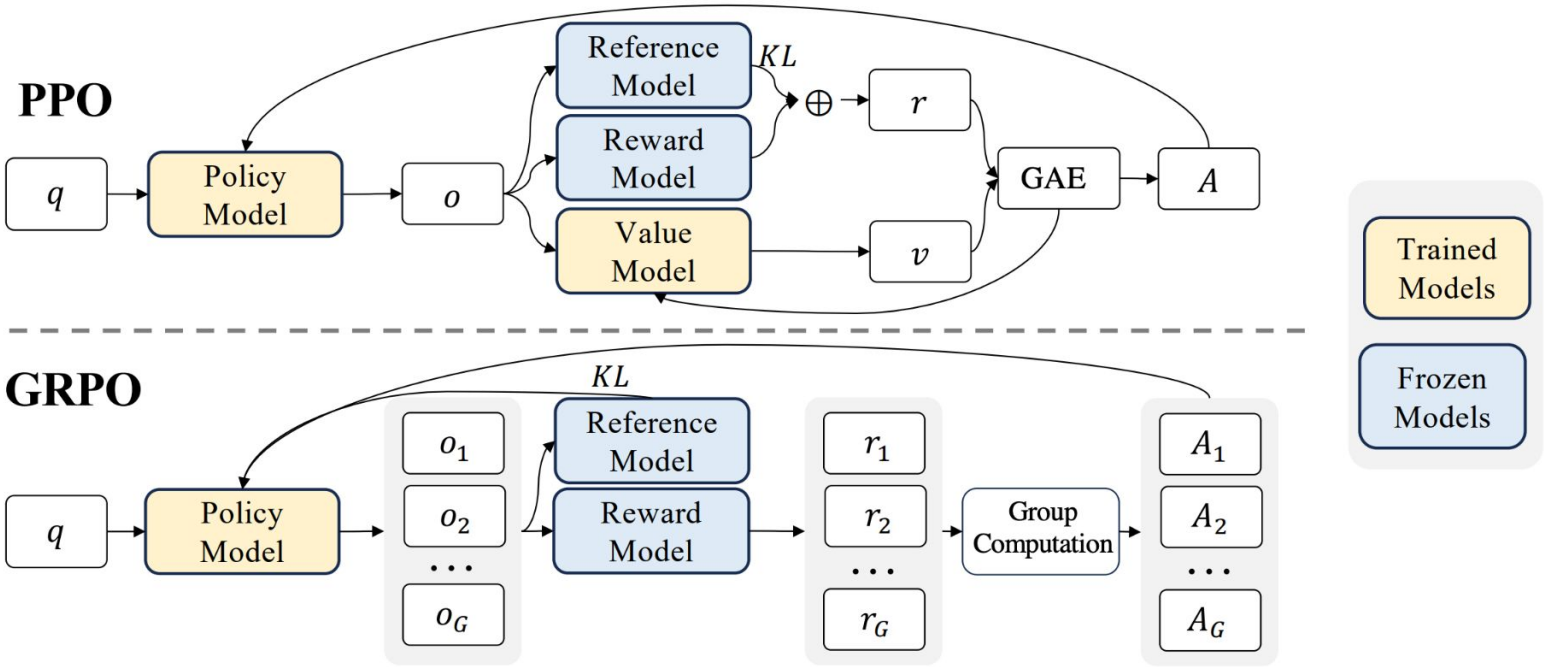
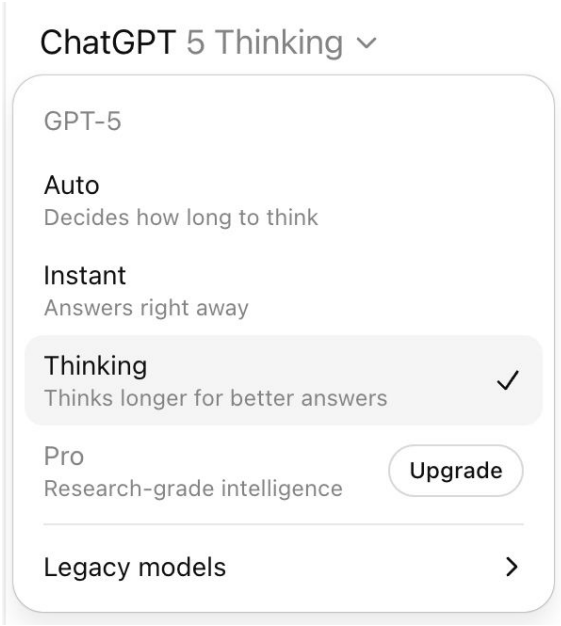
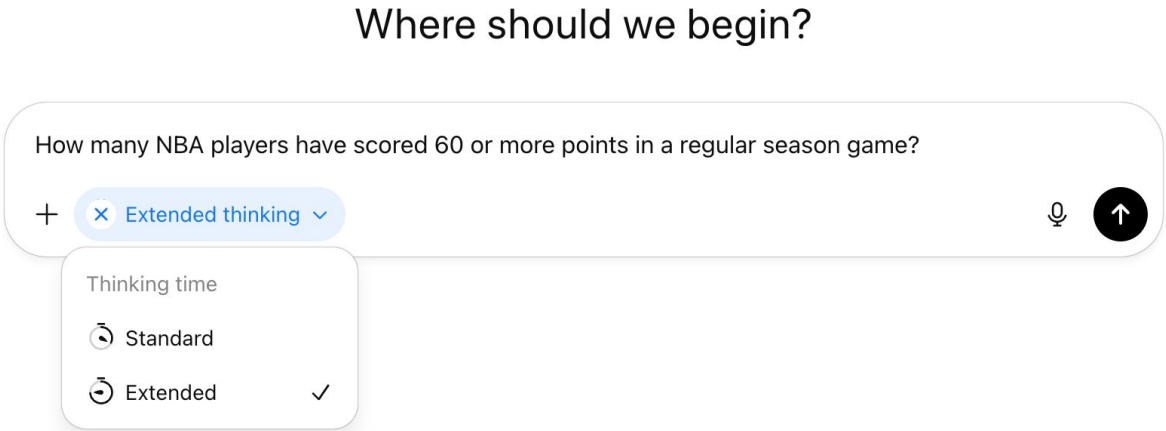
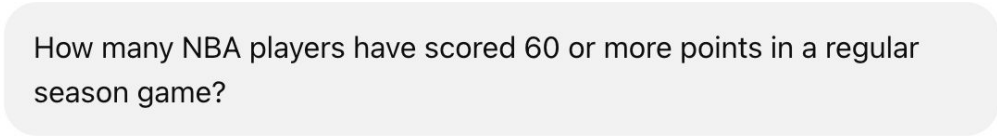


Figure 4 | Demonstration of PPO and our GRPO. GRPO foregoes the value model, instead estimating the baseline from group scores, significantly reducing training resources.

Large reasoning models (LRMs) / Thinking models

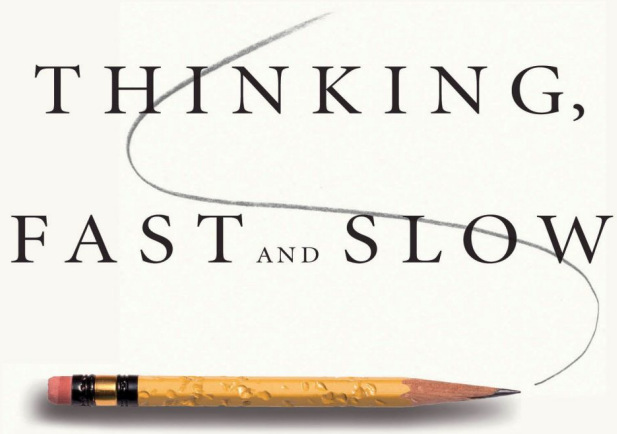








THINKING, FAST AND SLOW

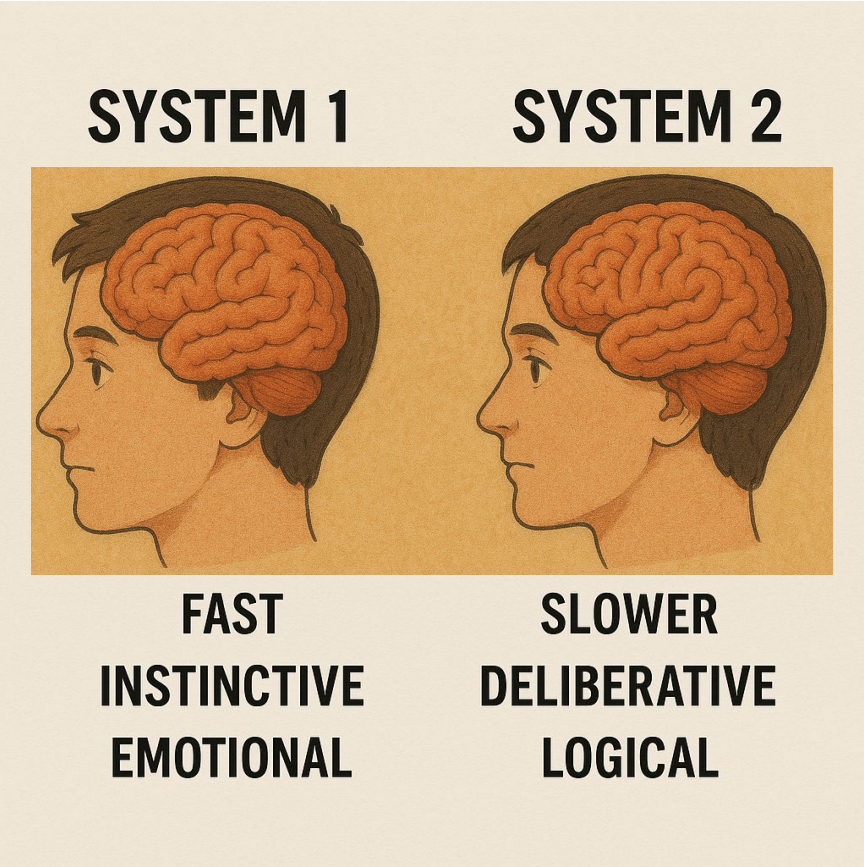


DANIEL KAHNEMAN

WINNER OF THE NOBEL PRIZE IN ECONOMICS

System 1 & System 2

Which is better?



ChatGPT image

OpenAI's o1

How it works

We trained these models to spend more time thinking through problems before they respond, much like a person would. Through training, they learn to refine their thinking process, try different strategies, and recognize their mistakes.

Similar to how a human may think for a long time before responding to a difficult question, o1 uses a chain of thought when attempting to solve a problem. Through reinforcement learning, o1 learns to hone its chain of thought and refine the strategies it uses. It learns to recognize and correct its mistakes. It learns to break down tricky steps into simpler ones. It learns to try a different approach when the current one isn't working. This process dramatically improves the model's ability to reason.

Can we infer its structure and what kinds of behaviors it rewards?

- The reward model likely provides scalar feedback that evaluates both **final answer** (correctness, usefulness, safety, clarity) and **reasoning process** (depth, coherence, reliability, safety awareness)
- Human annotators or automated evaluators review model outputs on reasoning tasks

What the reward model likely gives high scores for

1. Correct reasoning and factual accuracy

Outputs that reach correct conclusions through logically consistent reasoning receive higher rewards.

- On math or logic tasks: correct answers with verifiable reasoning.
- On open-ended questions: responses that are factually accurate and well supported.

2. Coherent, interpretable reasoning chains

Even if the chain of thought is not visible, internal reasoning steps that are internally consistent and lead to stable answers are likely rewarded.

- The model learns to reason step by step rather than jump to conclusions.
- RL training can penalize incoherent or contradictory internal reasoning trajectories.

3. Efficiency and confidence calibration

The model may get higher reward when it uses an appropriate amount of "thinking" (not too short or too long) and when it expresses uncertainty realistically.

- Correct but overly long reasoning could be slightly penalized.
- Overconfident incorrect answers could get low reward.

What the reward model likely gives high scores for (cont'd)

4. Safety, compliance, and harmlessness

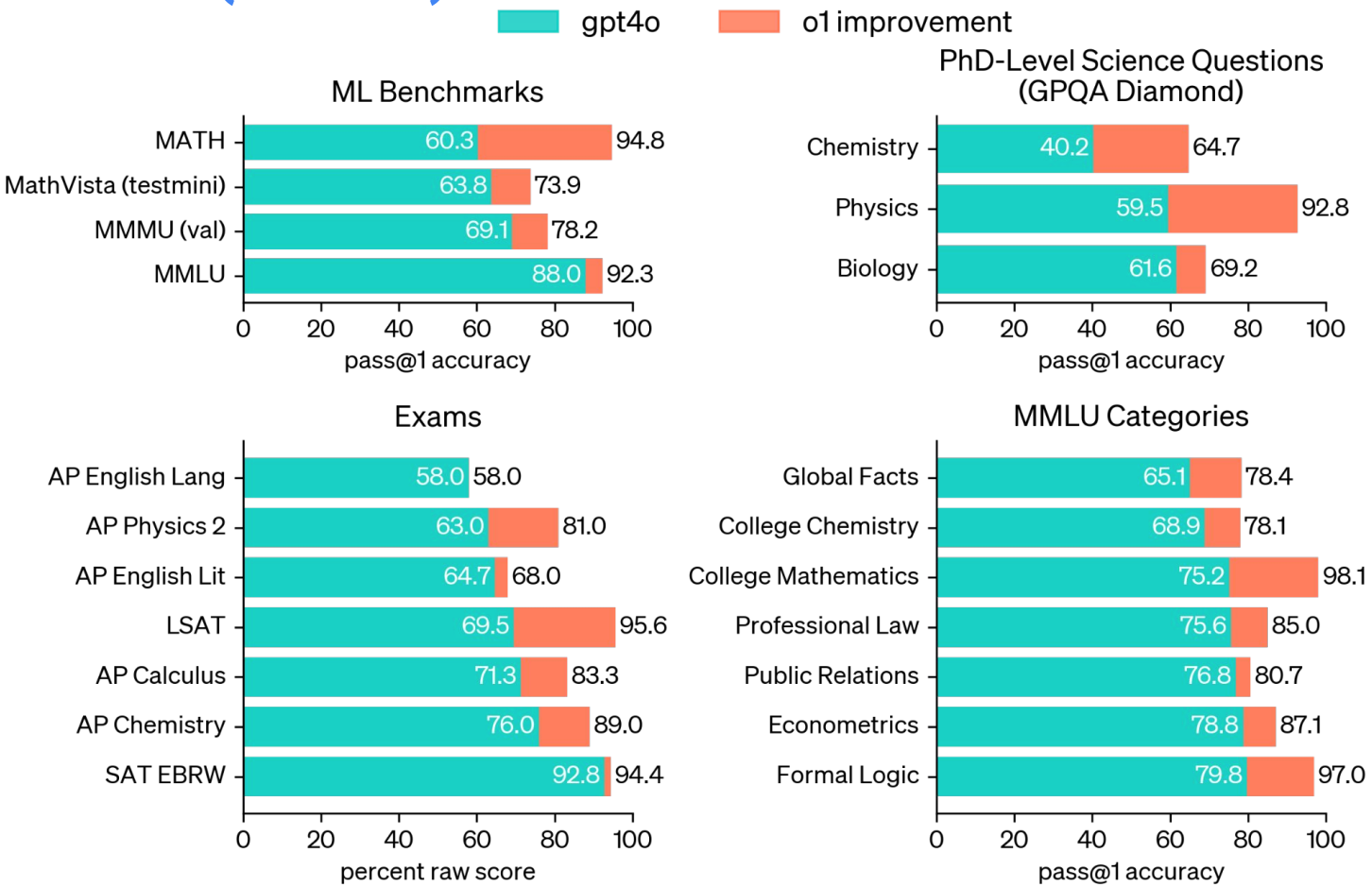
Outputs that adhere to safety policies and avoid unsafe or biased reasoning receive higher reward.

- The o1 system card emphasizes "deliberative alignment," meaning the model is rewarded for reasoning *about* safety before answering.
- Unsafe or policy-violating reasoning steps are penalized.

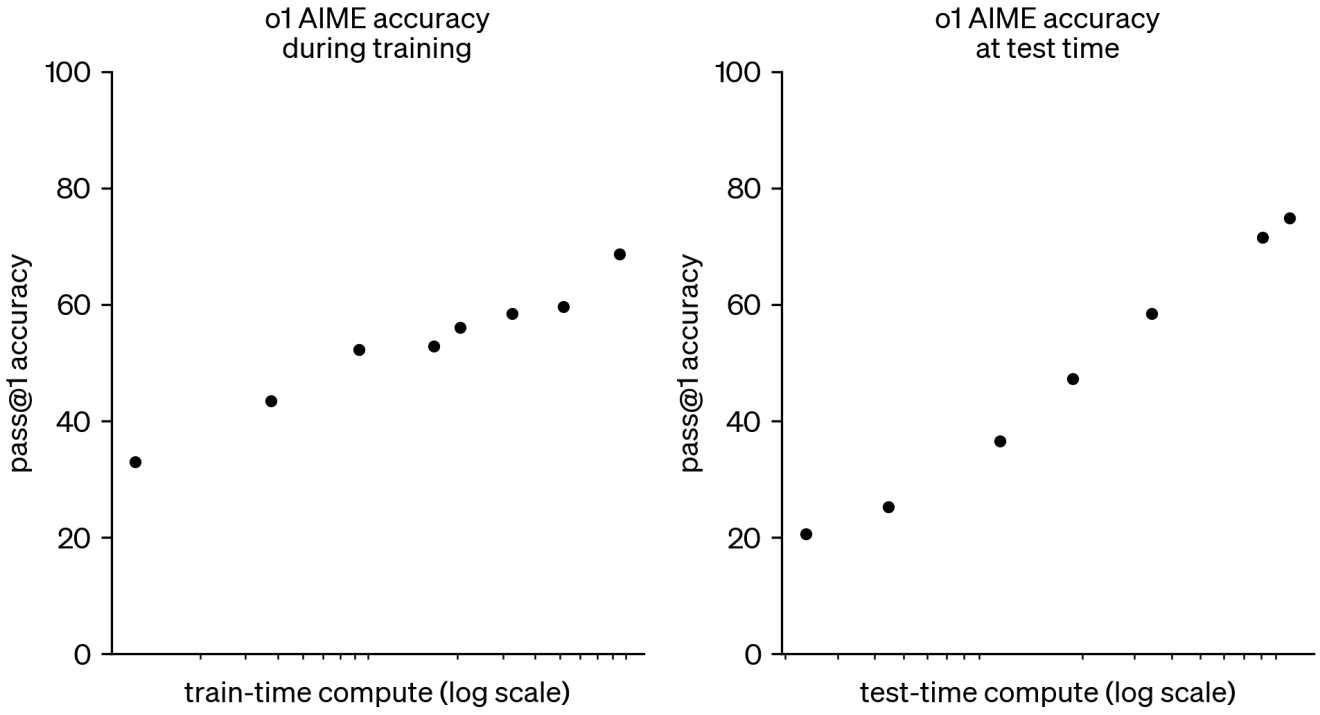
5. General helpfulness and clarity

The reward model also encourages clarity, helpful tone, and clear communication of reasoning results, similar to prior RLHF models.

OpenAI's o1 (cont'd)

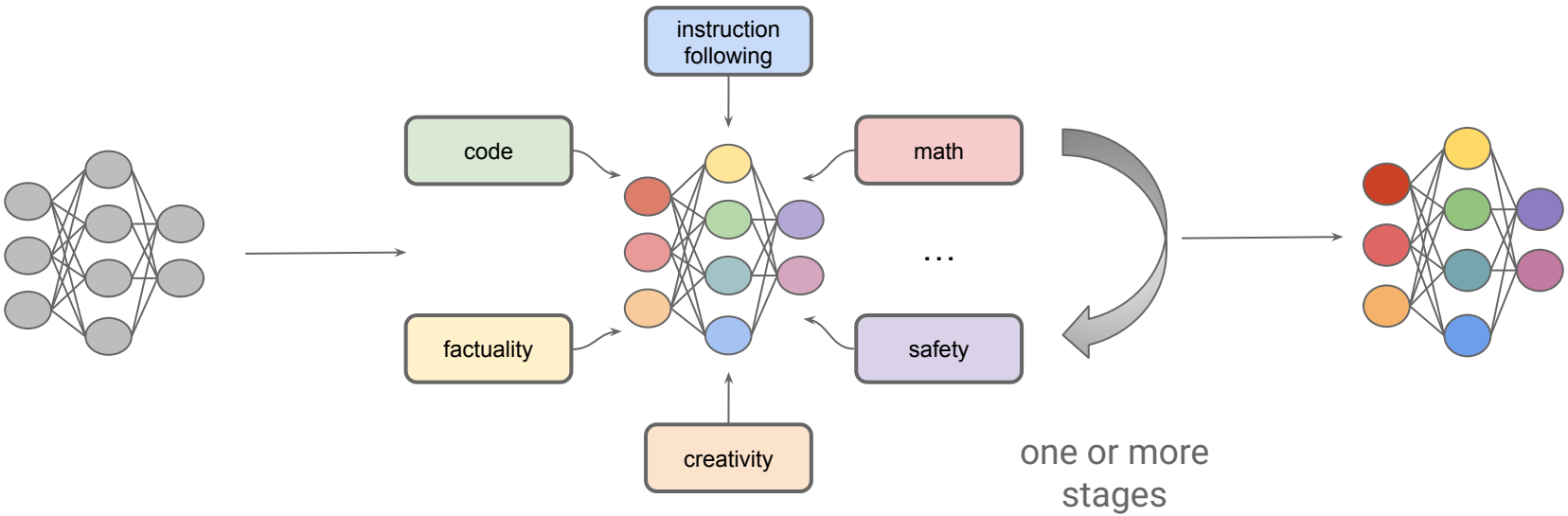


o1 performance smoothly improves with both train-time and test-time compute



https://openai.com/index/learning-to-reason-with-llms/

From System 1 to System 2



System 1

Supervised Fine-tuning and/or Reinforcement Learning on long Chain-of-Thought data

System 2

arXiv:2501.12948v1 [cs.CL] 22 Jan 2025

DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning

DeepSeek-AI

research@deepseek.com

Abstract

We introduce our first-generation reasoning models, DeepSeek-R1-Zero and DeepSeek-R1. DeepSeek-R1-Zero, a model trained via large-scale reinforcement learning (RL) without supervised fine-tuning (SFT) as a preliminary step, demonstrates remarkable reasoning capabilities. Through RL, DeepSeek-R1-Zero naturally emerges with numerous powerful and intriguing reasoning behaviors. However, it encounters challenges such as poor readability, and language mixing. To address these issues and further enhance reasoning performance, we introduce DeepSeek-R1, which incorporates multi-stage training and cold-start data before RL. DeepSeek-R1 achieves performance comparable to OpenAI-o1-1217 on reasoning tasks. To support the research community, we open-source DeepSeek-R1-Zero, DeepSeek-R1, and six dense models (1.5B, 7B, 8B, 14B, 32B, 70B) distilled from DeepSeek-R1 based on Qwen and Llama.

DeepSeek-R1’s motivation

- develop reasoning capabilities **without any supervised data (labeled input-output pairs)**
- focus on their **self-evolution** through **a pure reinforcement learning process**

Reinforcement Learning Algorithm

- **Group Relative Policy Optimization** (GRPO)
  - Sample outputs from the current (old) policy and then optimize the policy model by maximizing the scalar reward given by a reward model

Reinforcement Learning from Verifiable Rewards (RLVR)

The reward is the source of the training signal, which decides the optimization direction of RL. To train DeepSeek-R1-Zero, we adopt a rule-based reward system that mainly consists of two types of rewards:

- **Accuracy rewards**: The accuracy reward model evaluates whether the response is correct. For example, in the case of math problems with deterministic results, the model is required to provide the final answer in a specified format (e.g., within a box), enabling reliable rule-based verification of correctness. Similarly, for LeetCode problems, a compiler can be used to generate feedback based on predefined test cases.
- **Format rewards**: In addition to the accuracy reward model, we employ a format reward model that enforces the model to put its thinking process between '<think>' and '</think>' tags.

Guided Chain-of-Thought (CoT) template

A conversation between User and Assistant. The user asks a question, and the Assistant solves it. The assistant first thinks about the reasoning process in the mind and then provides the user with the answer. The reasoning process and answer are enclosed within <think> </think> and <answer> </answer> tags, respectively, i.e., <think> reasoning process here </think> <answer> answer here </answer>. User: prompt. Assistant:

Group Relative Policy Optimization (GRPO)

Group Relative Policy Optimization In order to save the training costs of RL, we adopt Group Relative Policy Optimization (GRPO) (Shao et al., 2024), which foregoes the critic model that is typically the same size as the policy model, and estimates the baseline from group scores instead. Specifically, for each question q, GRPO samples a group of outputs $\{o_1, o_2, \cdots, o_G\}$ from the old policy $\pi_{\theta_{old}}$ and then optimizes the policy model π_θ by maximizing the following objective:

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(O|q)]$$

$$\frac{1}{G}\sum_{i=1}^{G}\left(\min\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}A_i, \text{clip}\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1-\varepsilon, 1+\varepsilon\right)A_i\right) - \beta\mathbb{D}_{KL}\left(\pi_\theta||\pi_{ref}\right)\right), \tag{1}$$

$$\mathbb{D}_{KL}\left(\pi_\theta||\pi_{ref}\right) = \frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - \log\frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - 1, \tag{2}$$

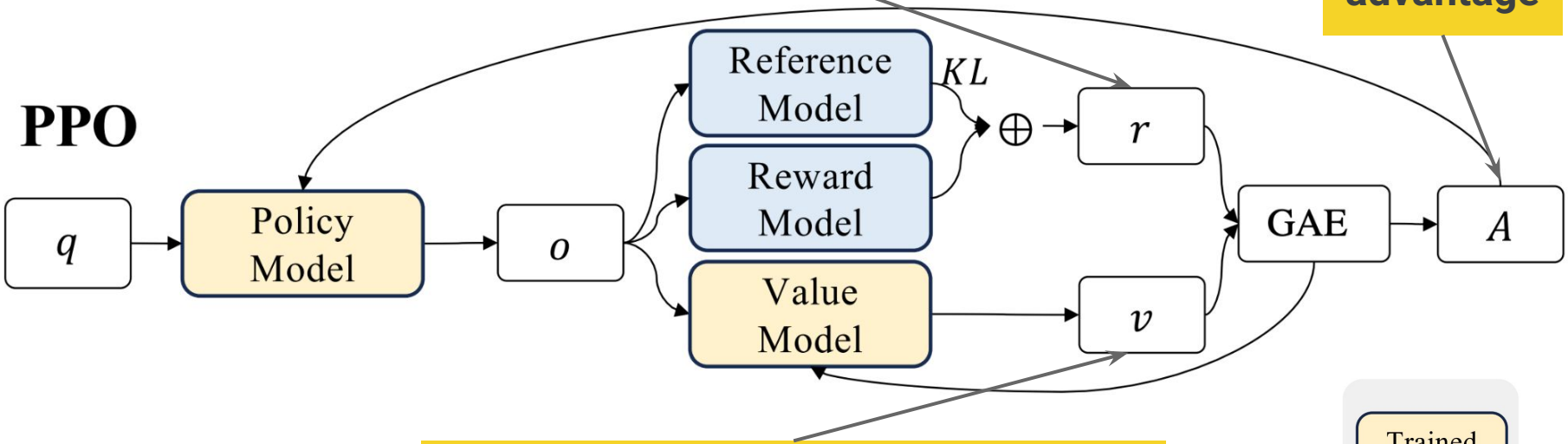
where ε and β are hyper-parameters, and A_i is the advantage, computed using a group of rewards $\{r_1, r_2, \ldots, r_G\}$ corresponding to the outputs within each group:

$$A_i = \frac{r_i - mean(\{r_1, r_2, \cdots, r_G\})}{std(\{r_1, r_2, \cdots, r_G\})}. \tag{3}$$

PPO

Think of the reward model as a teacher who grades each essay you write

advantage



The critic is like your own internal expectation ("I usually get an 80 on essays like this")

Group Relative Policy Optimization (GRPO) (cont'd)

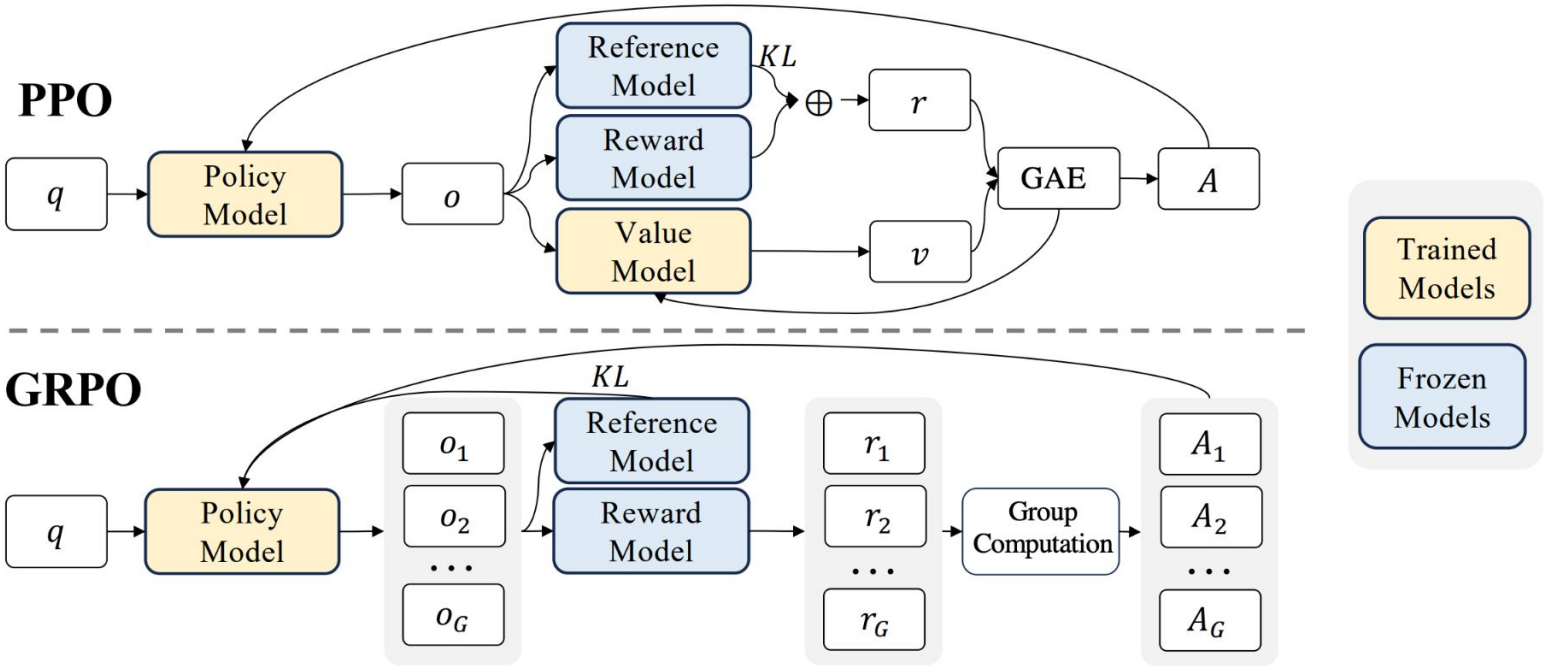


Figure 4 | Demonstration of PPO and our GRPO. GRPO foregoes the value model, instead estimating the baseline from group scores, significantly reducing training resources.

Group advantage estimation

The relative advantage for each output o_i is defined as:

$$A_i = \frac{r_i - \text{mean}(\{r_1, \ldots, r_G\})}{\text{std}(\{r_1, \ldots, r_G\})}.$$

This means that an output is considered “good” if its reward is higher than the group average.

This relative normalization stabilizes updates and avoids the need for an explicit baseline network.

Maximizing expected reward

The primary objective is to make outputs with higher rewards (from a reward model or human feedback) more likely under the updated policy.

This is achieved through the ratio term

$$\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{\text{old}}}(o_i|q)} A_i,$$

which increases the probability of samples with positive advantages $A_i > 0$ and decreases it for those with negative advantages $A_i < 0$.

Thus, GRPO pushes the model to favor responses that receive relatively higher scores within each group.

Maintaining policy stability

The **clipping function**

$$\text{clip}\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{\text{old}}}(o_i|q)}, 1-\epsilon, 1+\epsilon\right)$$

prevents large updates that could destabilize the model.

This is inherited from PPO: it ensures that the new policy does not deviate too aggressively from the previous one in a single training step.

Controlling divergence from the reference model

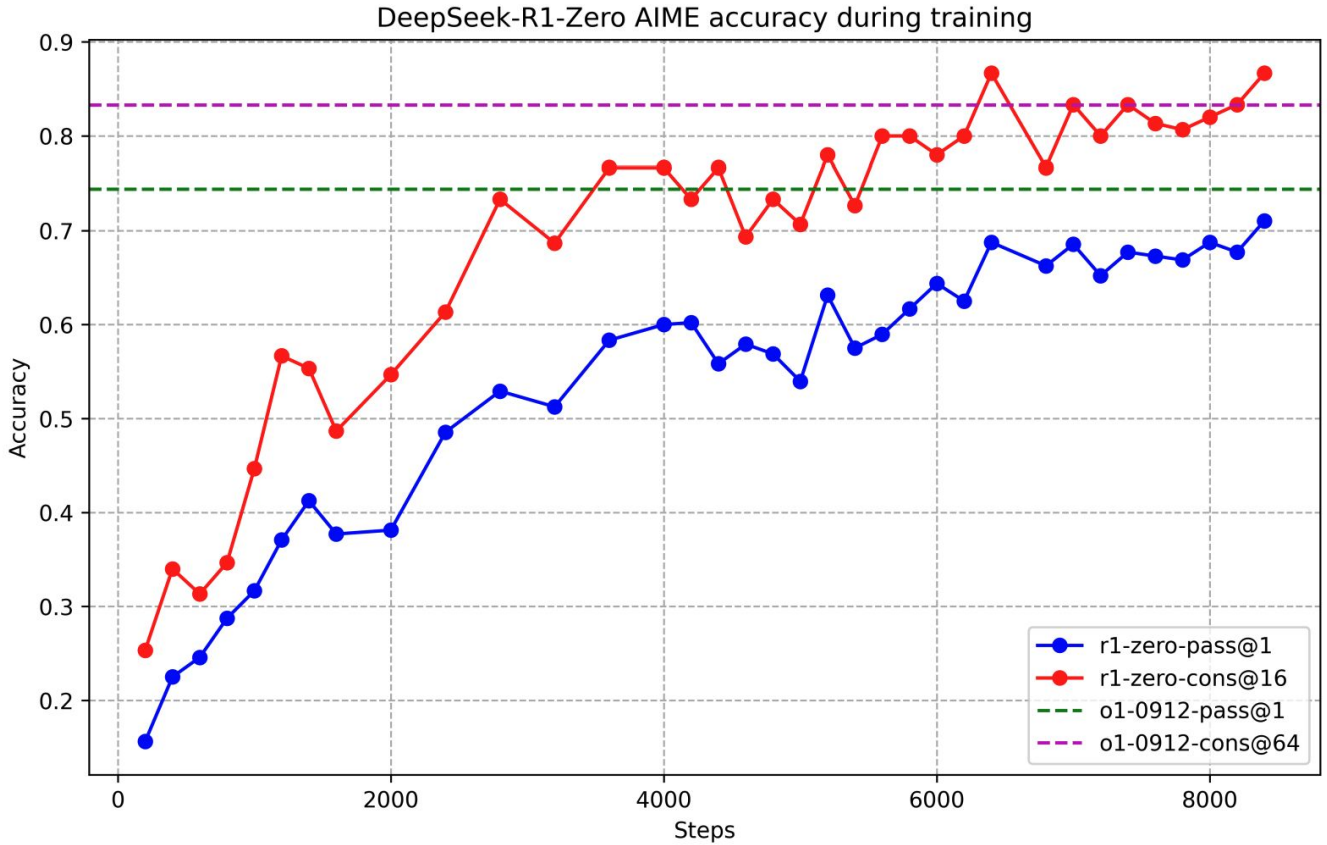
The **KL penalty term**

$$\beta D_{\mathrm{KL}}(\pi_\theta \| \pi_{\mathrm{ref}})$$

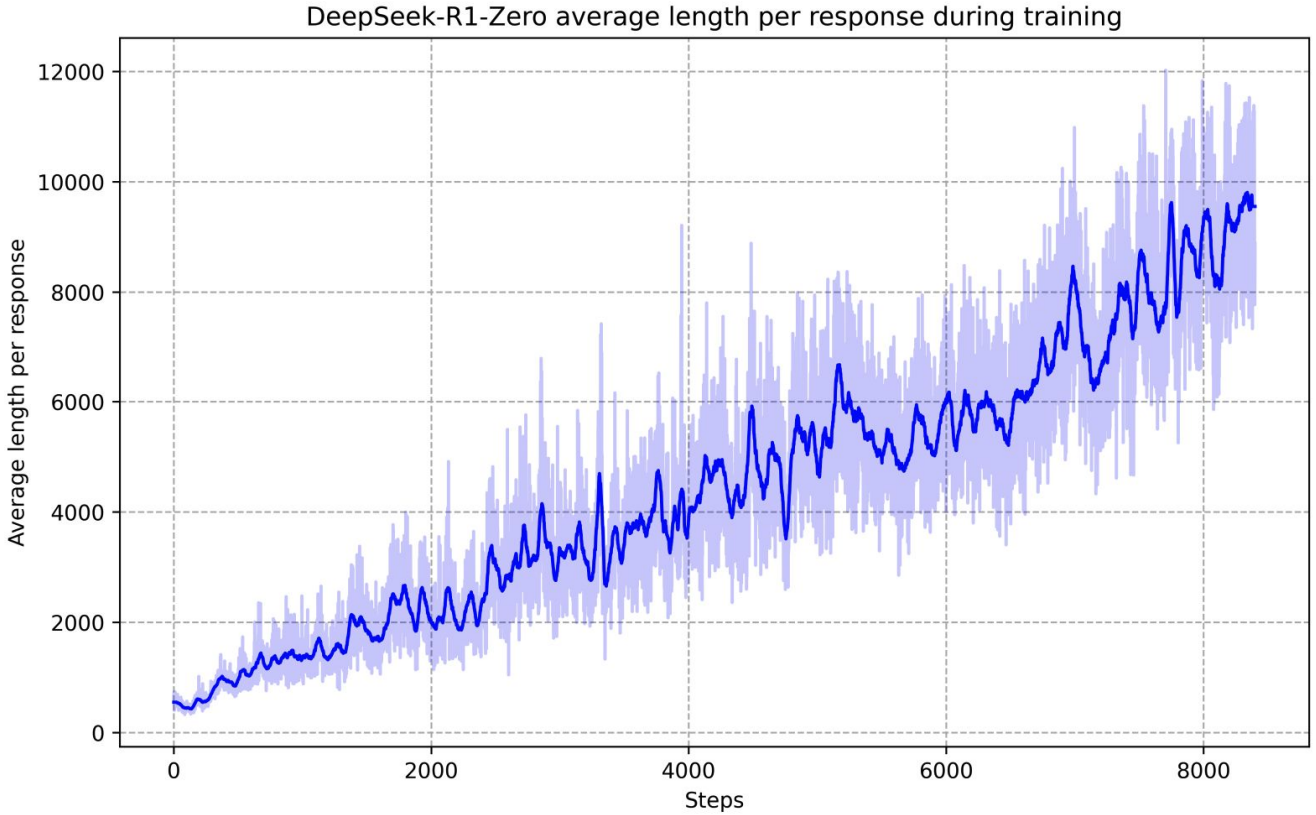
discourages the policy from diverging too far from a stable reference (often the supervised fine-tuned model).

This constraint keeps the learned behavior aligned with the base model's general language ability and prevents reward hacking or degeneration.

Accuracy of DeepSeek-R1-Zero during training



DeepSeek-R1-Zero naturally learns to solve reasoning tasks with more thinking time



An interesting “aha moment” of DeepSeek-R1-Zero

Question: If $a > 1$, then the sum of the real solutions of $\sqrt{a - \sqrt{a + x}} = x$ is equal to

Response: <think>

To solve the equation $\sqrt{a - \sqrt{a + x}} = x$, let's start by squaring both $\cdots$

$\left(\sqrt{a - \sqrt{a + x}}\right)^2 = x^2 \implies a - \sqrt{a + x} = x^2.$

Rearrange to isolate the inner square root term:

$(a - x^2)^2 = a + x \implies a^2 - 2ax^2 + (x^2)^2 = a + x \implies x^4 - 2ax^2 - x + (a^2 - a) = 0$

$\cdots$

Wait, wait. Wait. That's an aha moment I can flag here.

Let's reevaluate this step-by-step to identify if the correct sum can be $\cdots$

We started with the equation:

$\sqrt{a - \sqrt{a + x}} = x$

First, let's square both sides:

$a - \sqrt{a + x} = x^2 \implies \sqrt{a + x} = a - x^2$

Next, I could square both sides again, treating the equation: $\cdots$

$\cdots$

Problems with DeepSeek-R1-Zero

- It encounters challenges such as poor readability, and language mixing

Problems with DeepSeek-R1-Zero

- It encounters challenges such as poor readability, and language mixing

DeepSeek-R1

- Includes a small amount of cold-start data (thousands of long Chain-of-Thought (CoT) examples)

Post

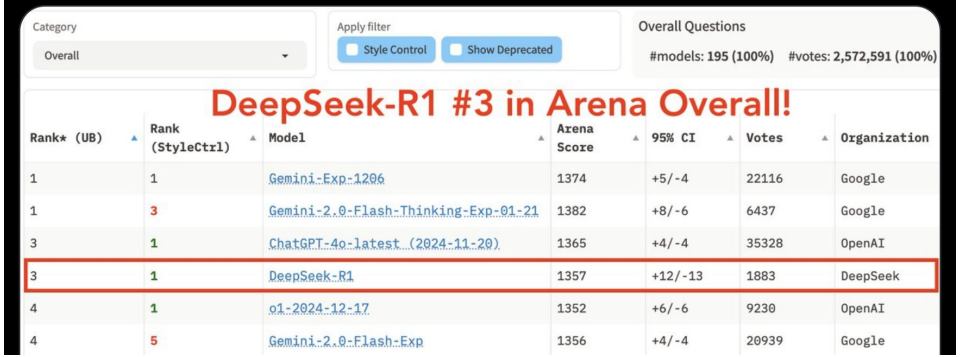
lmarena.ai (formerly lmsys.org)
@lmarena_ai

Breaking News: DeepSeek-R1 surges to the top-3 in Arena🐳!

Now ranked #3 Overall, matching the top reasoning model, o1, while being 20x cheaper and open-weight!

Highlights:
- #1 in technical domains: Hard Prompts, Coding, Math
- Joint #1 under Style Control
- MIT-licensed

A massive congrats to @deepseek_ai for this incredible milestone and gift to the community! More analysis below 👇



Rank* (UB)	Rank (StyleCtrl)	Model	Arena Score	95% CI	Votes	Organization
1	1	Gemini-Exp-1206	1374	+5/-4	22116	Google
1	3	Gemini-2.0-Flash-Thinking-Exp-01-21	1382	+8/-6	6437	Google
3	1	ChatGPT-4o-latest (2024-11-20)	1365	+4/-4	35328	OpenAI
3	1	DeepSeek-R1	1357	+12/-13	1883	DeepSeek
4	1	o1-2024-12-17	1352	+6/-6	9230	OpenAI
4	5	Gemini-2.0-Flash-Exp	1356	+4/-4	20939	Google

Thank you!